Su sistema ya ve.



Prof. : Manuela Larrea Gomez

Topicos Avanzados de Inteligencia Artificial

INTEGRAR MEDIR ENTREGAR

La sesión de hoy

- Recap: resolvemos las dudas del formulario de salida
- Dónde estamos · los tres patrones: decisión final y la receta de integración
- Lab parte 1: construir ver() y enchufarlo a su sistema
- Lab parte 2: casos con imagen, harness y tabla de delta contra M3
- Entrega M4 (cierra dom 25 OCT) · ética del contenido visual · hacia S14 (eficiencia)

Primero,
sus dudas.

Cuatro temas del formulario: las flechitas y los schedulers (DPM-Solver, Euler) · cómo se conectan las piezas de Stable Diffusion y ControlNet · "¿solo analizar, sin generar?" · los diez años: de carísimo a un Colab gratis.

SUS DUDAS 1/4 · LAS FLECHITAS Y LOS SCHEDULERS

Alguien preguntó por las flechas y los schedulers.

Con mucho gusto. Formalmente:

forward SDE (VP) · Song et al. 2021

$$d\mathbf{x} = -\tfrac{1}{2}\beta(t)\,\mathbf{x}\,dt + \sqrt{\beta(t)}\,d\mathbf{w}, \quad q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\,\mathbf{x}_0,\ (1-\bar\alpha_t)\mathbf{I}\big), \quad \bar\alpha_t = \prod_{s\le t}(1-\beta_s)$$

reverse-time SDE · Anderson 1982

$$d\mathbf{x} = \big[-\tfrac{1}{2}\beta(t)\mathbf{x} - \beta(t)\,\nabla_{\mathbf{x}}\log p_t(\mathbf{x})\big]dt + \sqrt{\beta(t)}\,d\bar{\mathbf{w}}$$

score ≡ ruido predicho · denoising score matching · Vincent 2011

$$\nabla_{\mathbf{x}}\log p_t(\mathbf{x}_t) = -\frac{\epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{1-\bar\alpha_t}}, \quad \mathcal{L}(\theta) = E_{t,\mathbf{x}_0,\epsilon}\big[\lambda(t)\,\|\epsilon - \epsilon_\theta(\sqrt{\bar\alpha_t}\,\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\,\epsilon,\ t)\|_2^2\big]$$

probability-flow ODE (misma marginal, sin azar)

$$\frac{d\mathbf{x}}{dt} = -\tfrac{1}{2}\beta(t)\mathbf{x} - \tfrac{1}{2}\beta(t)\,\nabla_{\mathbf{x}}\log p_t(\mathbf{x}) \;=\; F_\theta(\mathbf{x}, t)$$



DDPM · Ho et al. 2020 (ancestral, con azar)

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon_\theta(\mathbf{x}_t, t)\right) + \sigma_t\,\mathbf{z}, \quad \mathbf{z}\sim\mathcal{N}(0,\mathbf{I})$$

DDIM · Song et al. 2021 (σ_t = 0 → determinista)

$$\mathbf{x}_{t-1} = \sqrt{\bar\alpha_{t-1}}\left(\frac{\mathbf{x}_t - \sqrt{1-\bar\alpha_t}\,\epsilon_\theta}{\sqrt{\bar\alpha_t}}\right) + \sqrt{1-\bar\alpha_{t-1}-\sigma_t^2}\;\epsilon_\theta + \sigma_t\,\epsilon_t$$

Euler (orden 1 sobre la ODE)

$$\mathbf{x}_{t_{i+1}} = \mathbf{x}_{t_i} + (t_{i+1} - t_i)\,F_\theta(\mathbf{x}_{t_i}, t_i)$$

DPM-Solver-1 · Lu et al. 2022 · λ_t = log(α_t/σ_t), h_i = λ_{t_i} − λ_{t_{i−1}}

$$\mathbf{x}_{t_i} = \tfrac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\,\mathbf{x}_{t_{i-1}} - \alpha_{t_i}\int_{\lambda_{t_{i-1}}}^{\lambda_{t_i}} e^{-\lambda}\,\hat\epsilon_\theta(\hat{\mathbf{x}}_\lambda, \lambda)\,d\lambda \approx \tfrac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\,\mathbf{x}_{t_{i-1}} - \sigma_{t_i}(e^{h_i}-1)\epsilon_\theta(\mathbf{x}_{t_{i-1}}, t_{i-1})$$

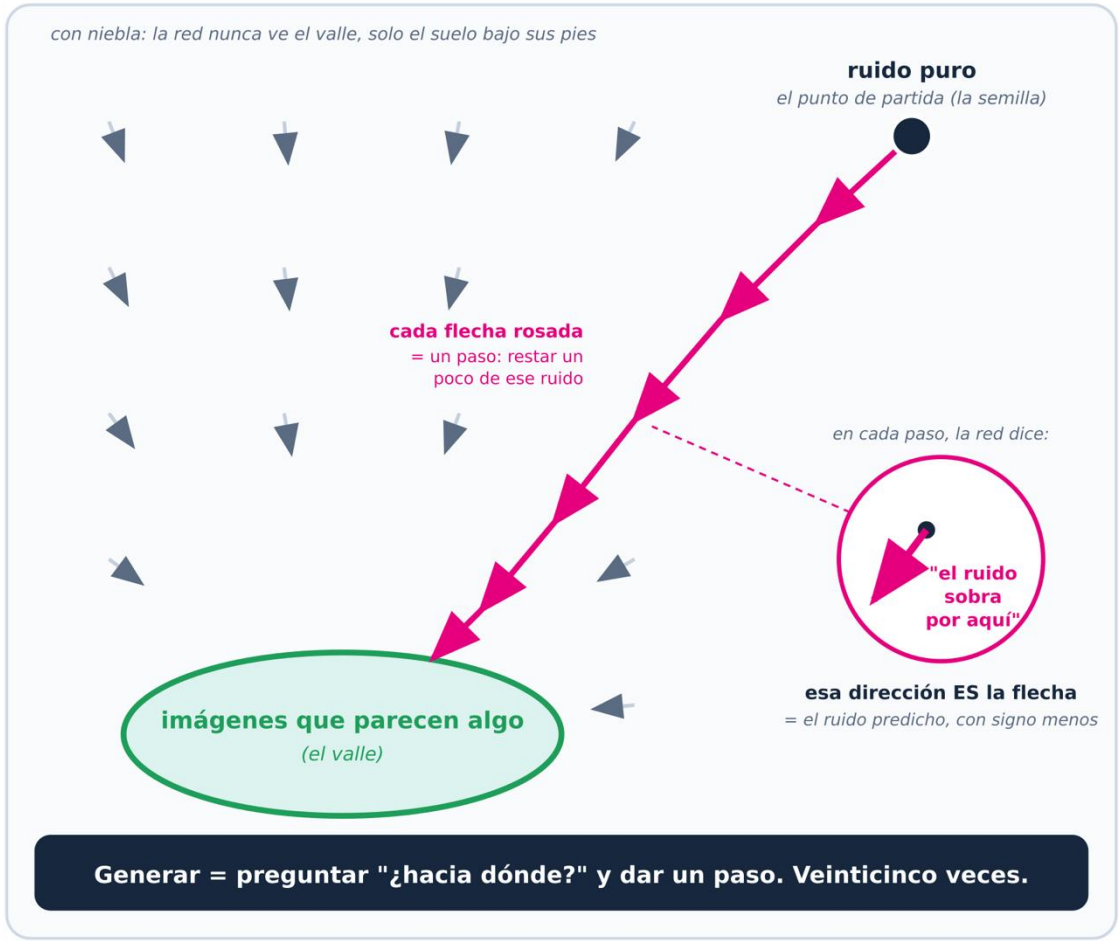
classifier-free guidance · Ho & Salimans 2022

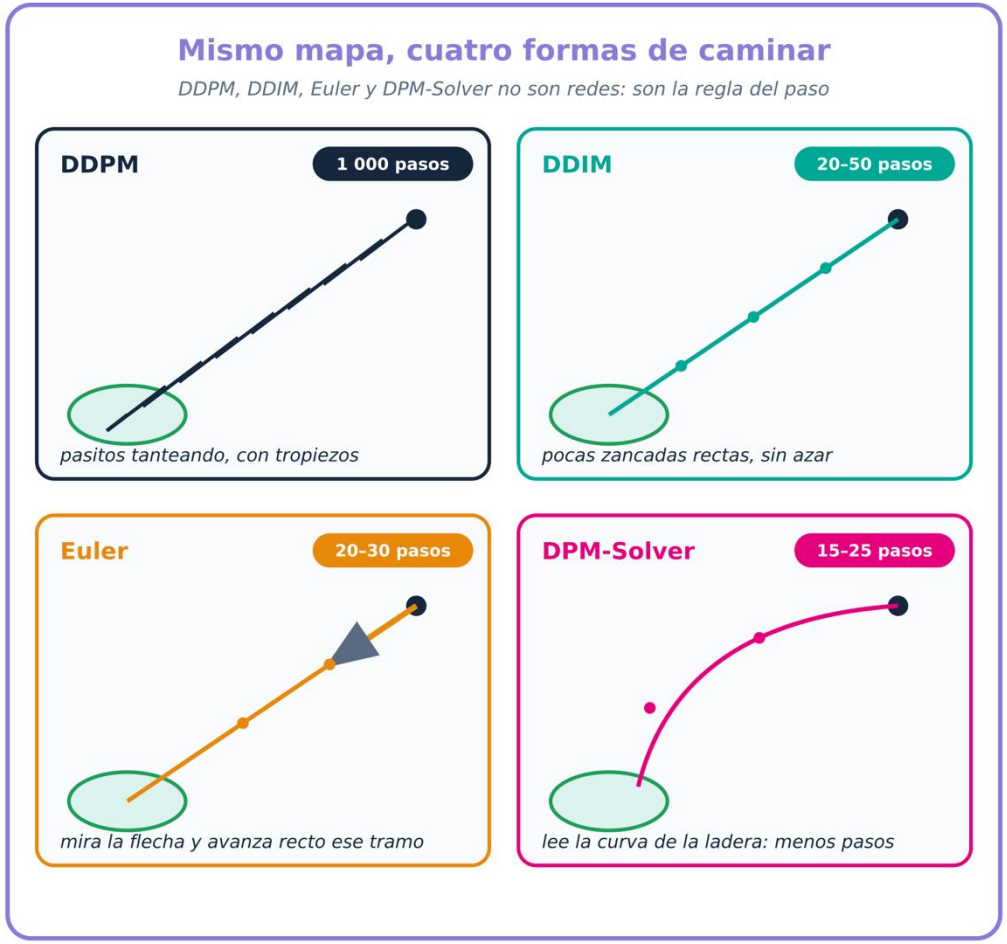
$$\tilde\epsilon_\theta(\mathbf{x}_t, \mathbf{c}) = \epsilon_\theta(\mathbf{x}_t, \emptyset) + s[\epsilon_\theta(\mathbf{x}_t, \mathbf{c}) - \epsilon_\theta(\mathbf{x}_t, \emptyset)]$$

¿preguntas?

La red no sabe dónde está la imagen. Solo sabe hacia dónde queda.

como bajar una montaña con niebla: no ves el valle, pero el suelo te dice hacia dónde es cuesta abajo





La red dibuja el mapa · el scheduler decide cómo caminar · ustedes eligen cuántos pasos

en el lab A de S12 cambiaron el número de pasos y midieron segundos: eso era exactamente esto

Stable Diffusion: tres máquinas y un reloj — siguiendo UNA imagen

qué le pasa a la foto del cachorro en cada estación (y cuántas veces trabaja cada máquina)

× 1

1 · EL TRADUCTOR

trabaja una vez, al principio

"a beagle puppy looking up, sitting on a carpet"

↓

tokenizer + text encoder

CLIP, el de S11 · congelado

↓

77 vectores



Fíjense: aquí NO hay imagen todavía.

Solo números que dicen qué quiere el texto.
Esos números se le prestan a la máquina 2.

× 25

2 · EL QUE LIMPIA + EL RELOJ

veinticinco vueltas, siempre en pequeño (64 × 64)

los 77 vectores entran en CADA vuelta

U-Net
"este ruido sobra" (la flecha)

→ **reloj**
cuánto restar

lo que la U-Net ve en cada vuelta: el mismo cuadro, cada vez un poquito más limpio



vuelta 0 *ruido puro* · **vuelta 7** · **vuelta 14** · **vuelta 20** · **vuelta 25** *latente limpio*

Todo esto pasa en PEQUEÑO y pixelado: 64 × 64, no 512 × 512.
Por eso cabe en una T4. Todavía no es la foto: es el "negativo".



ControlNet (opcional)
con los bordes de una foto de referencia, le sopla la FORMA a la U-Net en cada vuelta. El resto no cambia.

× 1

3 · EL REVELADOR

trabaja una vez, al final



latente limpio · 64 × 64

↓

decoder del VAE

de pequeño a grande: "revela" la foto

↓



imagen 512 × 512 · la foto

texto una vez → limpiar 25 veces, en pequeño (U-Net + reloj) → revelar una vez

el scheduler no aprende nada: es la regla del paso · ControlNet no cambia el bucle: solo le suma forma a la U-Net

Honestidad del dibujo: en un modelo real el "negativo" tiene 4 canales y no se ve así de parecido a la foto; lo mostramos en color para que se reconozca el cachorro.

StableDiffusionPipeline.from_pretrained(...) trae las tres máquinas y el reloj adentro.

¿Solo analizar imágenes, sin generar? Sí. Es lo más común.

una imagen puede ENTRAR a su sistema o SALIR de él — tres casos que ya conocen

LA IMAGEN ENTRA

1 · ENTRA Y LA LEE · patrón INPUT

Be My Eyes + GPT-4V (2023) · "mirar" la nevera



→ **VLM** *lee la imagen (S11)* → **texto** *"hay yogur, tomates y huevos: puedes hacer una tortilla"*

alguien manda la foto de su nevera y pregunta. Entra imagen, sale texto. No se generó nada.

2 · ENTRA Y LA CLASIFICA · patrón TOOL

Cake or Fake · el reto viral



→ **CLIP / ViT** *clasifica (S11)* → **una clase** *"cake" · "corn" hoy: roya / sana · dañado / ok*

¿torta o maíz? un clasificador como herramienta: el agente lo llama cuando hay foto. Sale una etiqueta.

SU SISTEMA (M3)

retriever + LLM
+ tools + harness

no se reescribe:
se le enchufa UNA cosa

LA IMAGEN SALE

3 · SALE: LA DIBUJA · patrón OUTPUT

El Papa con chaqueta Balenciaga · Midjourney (marzo 2023)

→ **difusión** *dibuja (S12)* →



millones la creyeron real. Es la única de las tres que GENERA, y por eso la etiqueta "imagen generada" no es opcional (S12).

para su proyecto: opcional, solo si dibujar responde mejor que escribir.

Para M4 basta UNO de los tres. La mayoría de los equipos está en 1 o 2: la imagen entra, se analiza, y lo que sale es texto o una clase que su sistema ya sabe usar.

Analizar sin generar no es la versión recortada: es el caso principal.

difusión es una opción, no un requisito · lo obligatorio es integrar UN componente y medirlo con ≥5 casos con imagen

Entrenar sigue siendo carísimo. Lo que se volvió barato es USAR.

como un restaurante: montar la cocina cuesta millones; pedir un plato cuesta lo que cuesta un plato

MONTAR LA COCINA = ENTRENAR

lo paga UNA empresa, UNA vez

Stable Diffusion: ~600 mil USD en GPU · **CLIP:** 256 GPUs, dos semanas

GPT-4, Flux, SD 3: millones. **Esto NO bajó. Y no es su problema.**

Stability · OpenAI · Meta · Google pagan esta cuenta

PEDIR UN PLATO = USAR

lo paga cualquiera, millones de veces

from_pretrained() baja el modelo ya entrenado · corre en una T4 prestada

una imagen de SD: ~5 segundos, fracciones de centavo. **Esto SÍ bajó, 1 000×.**

ustedes viven aquí: from_pretrained + LoRA + Colab

¿Qué bajó el costo de USAR un modelo de visión? Cuatro escalones, una causa cada uno

costo de usar · escala ilustrativa

2012 — semanas de un lab con GPUs

un laboratorio

AlexNet: 2 GPUs, seis días, un equipo entero.

2015

reutilizar en vez de entrenar

ResNet + transfer learning: bajar pesos ya entrenados y ajustar.

2018

a Colab: que Dios lo bendiga



una GPU gratis en el navegador

Google Colab: la T4 que usan hoy, sin instalar nada.

2021–22

pesos abiertos

CLIP y Stable Diffusion publicados: cualquiera los baja.

2023 — una T4 gratis, 5 segundos por imagen

comprimir

4 bits, LoRA, destilación: lo grande cabe en una T4.

La pregunta de su proyecto nunca es "¿cómo entreno?" sino "¿cuál bajo, cómo lo adapto, y cuánto cuesta cada llamada?"

preentrenado + adaptación ligera (LoRA, S04) + inferencia: el curso entero vive del lado barato de la curva

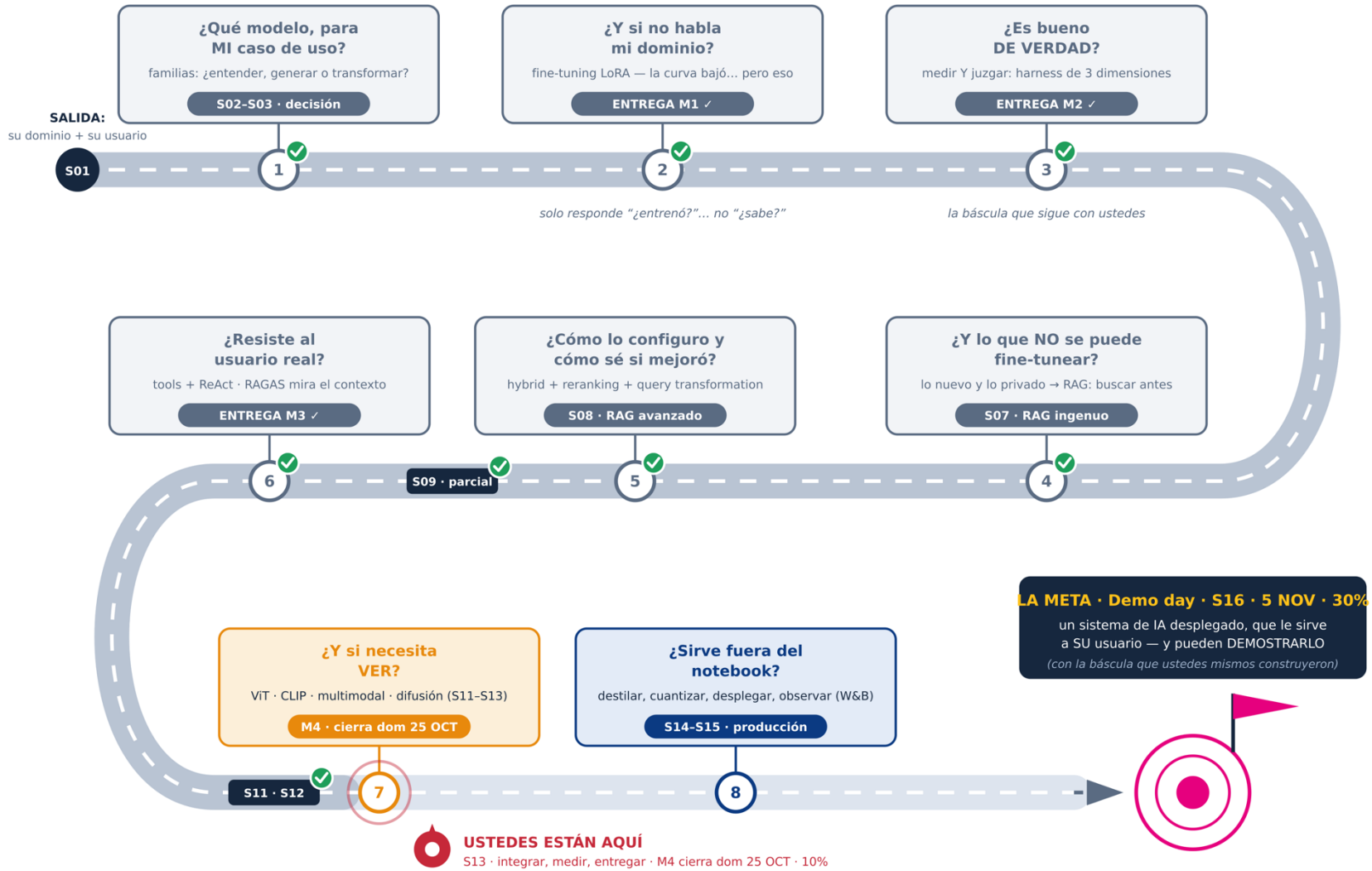
En S14 (Módulo 5) vemos el escalón de 2023 por dentro: destilación y cuantización. Es lo que hace que SU sistema también sea barato de usar.

¿Dónde estamos?

El mapa, actualizado: M3 entregado, S11 y S12 recorridas. Hoy se cierra la parada 7.

El mapa del semestre: hoy se cierra la parada 7 — el sistema ya ve, falta medirlo y entregarlo

lo recorrido en gris · lo que falta en color · la meta no cambió



Lo que falta: M4 cierra dom 25 OCT (10%) · S14 eficiencia · S15 despliegue · demo day 5 NOV (30%) + asistencia (10%) = M1+M2+M3+M4 + producción

Ya tienen todas las piezas. Hoy se ensamblan.

LO QUE YA TIENEN

Construido entre M2 y S12

Un sistema que responde (M3: RAG + tools + agente).

Un harness con eval set y scorecard (M2).

Dos ladrillos visuales: CLIP y un VLM (S11).

Un generador de imágenes con control (S12).

LO QUE FALTA

Tres cosas, hoy

UN componente visual enchufado al sistema, de punta a punta.

≥5 casos con imagen en el eval set, cada uno con su criterio.

La tabla de delta contra M3: ¿mejoró lo nuevo? ¿no empeoró lo viejo? ¿cuánto cuesta?

LO QUE SE ENTREGA

M4 · cierra dom 25 OCT · 10 %

Pipeline multimodal funcionando + casos de prueba + scorecard M4 vs. M3 + README de una página con la justificación del patrón.

Hoy terminan con el 80 %; el resto es pulir.

La pregunta de hoy no es "¿qué modelo?" (eso fue S11 y S12) sino "¿dónde lo enchufo, cómo sé que sirve, y cuánto cuesta?". Eso es integrar.

Elijan uno. Hoy, antes del lab.

	VISIÓN COMO INPUT	VISIÓN COMO TOOL	VISIÓN COMO OUTPUT
Quién trae la imagen	el usuario, con su pregunta	el agente la busca o la recibe de una tool	nadie: el sistema la produce
Qué hace ver()	VLM → texto o JSON	CLIP / ViT / SAM → clase, número, recorte	prompt → difusión → imagen
Dónde se enchufa	ANTES del RAG: la salida se suma a la pregunta	EN la lista de tools (S10): el LLM decide llamarla	DESPUÉS del LLM: la respuesta incluye la imagen
Cómo se mide	campos correctos · juez VLM · exacto	accuracy de la tool · ¿la llamó cuando debía?	checklist VLM · CLIP score · humano 1–5
Lo que más falla	alucina lo que no se ve → válvula	la llama cuando no toca (o no la llama)	inventa detalles → etiqueta "generada"

Para M4 basta UNO. Si dudan entre dos, elijan el que responde la pregunta de su usuario con menos piezas nuevas. Y si es "ninguno": justifíquenlo y midan el más barato.

Cinco pasos. Los dos labs de hoy los recorren.

1

LA PREGUNTA QUE NECESITA OJOS
Una frase, de su usuario. "¿Esta hoja tiene roya?" · "¿Qué dice este contrato escaneado?" · "¿Cómo se vería el sofá en mi sala?". Si no existe, el patrón es "ninguno".

2

LA FUNCIÓN ver()
imagen → texto / JSON (input) · imagen → clase o recorte (tool) · prompt → imagen (output). Una función, probada sola sobre 3 imágenes del dominio, con su latencia anotada.

3

EL ENCHUFE
Dónde entra al sistema de M3: antes del RAG, en la lista de tools o después del LLM. sistema_multimodal(pregunta, imagen) responde de punta a punta.

4

LOS CASOS CON IMAGEN
≥5 casos en el eval set con la imagen, lo esperado y el criterio (exacto, campos, válvula, juez VLM, checklist). Al menos uno adversarial.

5

EL HARNESS Y EL DELTA
La misma función de M2 califica los casos nuevos. La tabla: M3 vs. M4 en casos con imagen, sin imagen, y latencia.

El VLM traduce la imagen; el sistema de M3 no cambia.

EL ESQUELETO

ver() antes del RAG

```
def ver(imagen, instruccion):
    # Qwen2-VL (S11): describe / extrae JSON
    return preguntar(imagen, instruccion,
                     system=VALVULA)

def sistema_multimodal(pregunta, imagen):
    visto = ver(imagen, INSTRUCCION_DOMINIO)
    contexto = f"Lo que se ve: {visto}"
    return responder_m3(pregunta, contexto)
    # ↑ su RAG + agente de M3, sin tocar
```

LAS TRES REGLAS

Lo que ya saben, aplicado

Válvula de escape en la instrucción: "si no se aprecia, di que no se aprecia". Sin esto el VLM inventa la fecha, la placa o la raza (S11).

Salida estructurada y VALIDADA: pidan JSON con campos fijos y parseen con cuidado (lab A de S10). Si falla, el caso cuenta como fallo.

Describir antes de preguntar: dos llamadas (describe → responde) reducen alucinación. Cuestan el doble: midan si vale.

La imagen entra al sistema como texto. Por eso el RAG, el agente y el harness de M3 siguen funcionando igual: ver() es un traductor en la puerta.

La visión es una herramienta más; el LLM decide cuándo mirar.

EL ESQUEMA (S10)

Una tool con imagen

```
{"name": "clasificar_imagen",
 "description": "Clasifica la foto adjunta en
   una de: roya, minador, sana. Úsala solo
   si el usuario adjuntó una foto de hoja.",
 "parameters": {"imagen_id": "string"}}

def clasificar_imagen(imagen_id):
    # CLIP zero-shot (S11) o ViT + logística
    return {"clase": "roya", "coseno": 0.31}
```

LO QUE SE MIDE

Dos preguntas, no una

¿La tool acierta? accuracy sobre 20+ imágenes etiquetadas por ustedes. Es el eval set de la tool, aparte del eval set del sistema.

¿El agente la llama cuando debe (y no cuando no)? Casos con foto donde debió llamarla; casos sin foto donde no. Es la métrica de S10: ¿eligió la herramienta correcta?

Y la latencia: una llamada a CLIP es barata; a un VLM, no. Anótenla por caso.

La descripción de la tool es parte del modelo: "úsala solo si hay foto" cambia cuántas veces el agente la llama. Es un hiperparámetro, y se mide.

El sistema dibuja — y lo dice.

EL ESQUELETO

ver() después del LLM

```
def generar(prompt, seed=42):
    g = torch.Generator("cuda").manual_seed(seed)
    return pipe(prompt, generator=g,
                guidance_scale=7.5).images[0]

def sistema_multimodal(pregunta):
    texto = responder_m3(pregunta)
    prompt = plantilla_prompt(texto)   # inglés
    imagen = generar(prompt)
    return texto, etiquetar(imagen)   # "generada"
```

LO QUE SE MIDE

Tres semillas y una checklist

- CLIP score (S12): fidelidad al prompt, no calidad. Automático; promedien 3 semillas.
- Checklist con VLM: "¿aparece X? ¿estilo Y?" → ítems cumplidos / total. Es el juez de M2, pero mirando.
- Humano 1–5 en pocos casos: ¿le sirve al usuario?
- Y la válvula de salida: pedidos que el sistema NO debe dibujar (un caso clínico "real", una firma). Se mide: ¿se negó?

Si la figura debe ser EXACTA (geometría, un diagrama de proceso), la mejor "imagen generada" es código: Mermaid o matplotlib. Cuenta como output si lo justifican.

Al notebook · Parte 1

Construir ver() y enchufarla a su sistema

Eligen el patrón en la primera celda. Construyen ver() con el modelo que corresponde (VLM, CLIP o SD), la prueban sola sobre 3 imágenes de su dominio y anotan la latencia. Luego la enchufan: sistema_multimodal(pregunta, imagen) responde de punta a punta usando su responder_m3 (o el sustituto del notebook si aún no lo conectan).

PATRON = … → ver(imagen) → probar en 3 imágenes → latencia → sistema_multimodal(pregunta, imagen) · S13_Lab_Integracion_Visual.ipynb

Clase grabada: pausen el video aquí y trabajen la parte 1 con su equipo (~45 min). En el aula: 45 min y volvemos.

Antes de seguir, cuatro cosas deben ser ciertas.

- **ver() corre sola sobre 3 imágenes del dominio** — y la salida es lo que prometieron: texto, JSON parseable, una clase o una imagen. Si no, la parte 2 no tiene qué medir.
- **La latencia está anotada** — segundos por llamada, en la GPU o CPU donde la corrieron. Va al scorecard.
- **sistema_multimodal responde de punta a punta** — pregunta + imagen → respuesta. Con su responder_m3 real, o con el sustituto del notebook marcado como tal.
- **Ya vieron el primer fallo** — una imagen donde ver() se equivocó o alucinó. Anótenla: es su primer caso adversarial de la parte 2.

Si algo de esto no es cierto, no avancen: la parte 2 mide lo que la parte 1 construyó. Un pipeline que no corre no tiene scorecard.

La vara no cambia.

El harness de M2 califica el componente visual igual que calificó el RAG. Lo único nuevo: casos con imagen, y criterios que saben mirar.

Casos con imagen: las mismas columnas, un archivo más.

ID	INPUT (texto + imagen)	ESPERADO	CRITERIO	TIPO
img_01	"¿Qué enfermedad tiene esta hoja?" + hoja_roya.jpg	roya	exacto (clase)	normal
img_02	"Extrae los datos de este carnet" + carnet.jpg	{nombre, fecha, dosis}	campos correctos / 3	normal
img_03	"¿Qué fecha tiene este sello?" + foto_sin_sello.jpg	"no se aprecia en la imagen"	válvula (0/1)	adversarial
img_04	"Describe la lesión" + lesion.jpg	descripción gold (texto)	juez VLM: ¿se sostiene en la imagen?	normal
img_05	"Muéstrame el sofá en una sala pequeña"	sofá gris · sala pequeña · ventana	checklist VLM (x/3) + CLIP score	output

Mínimo 5, al menos uno adversarial. Las imágenes van en una carpeta del repo (anonimizadas, sin datos personales) y el CSV las referencia por nombre. Es el mismo eval set de M2, con una columna más.

Seis formas de calificar un caso con imagen.

exacto · campos
Para clases, números y JSON: ¿coincide? ¿cuántos campos de N? Automático, sin LLM. Los de M2, sin cambios.

válvula
El caso trampa: lo esperado es "no se aprecia" o "no puedo generar eso". 1 si se negó, 0 si inventó. Es el caso más barato y el que más fallos encuentra.

juez VLM
Faithfulness visual: un VLM (o un LLM con la descripción gold como contexto) decide si lo que dijo el sistema se sostiene en la imagen. RAGAS con otro contexto (S10).

checklist VLM · CLIP score
Para output: ítems cumplidos / total, y el coseno imagen–prompt × 100 promediado en 3 semillas (S12). Fidelidad, no calidad.

latencia
Segundos por caso, siempre. Un VLM en la ruta multiplica el tiempo de respuesta: el scorecard lo dice.

Regla de M2 que sigue viva: cada caso tiene UN criterio declarado antes de correr el sistema. Si eligen el criterio después de ver la respuesta, no es evaluación.

Tres preguntas, una tabla.

GRUPO DE CASOS	M3 (baseline)	M4 (con visión)	DELTA	LECTURA
casos sin imagen (los 10+ de M2)	0.74	0.73	−0.01	¿empeoró lo viejo? aquí, no
casos con imagen (≥5 nuevos)	0.00 (no podía)	0.68	+0.68	¿se responden ahora? sí, 4 de 5… ¿cuál no?
caso adversarial (válvula)	—	1 / 1	—	¿se negó cuando debía? sí
latencia media por caso	2.1 s	6.4 s	+4.3 s	¿cuánto cuesta ver? tres veces más
peor caso	q_07 (0.31)	img_04 (0.20)	—	¿dónde falla? el juez no sostuvo la descripción

Números inventados para el ejemplo. Los suyos salen del notebook. Lo que se califica no es que M4 gane: es que la tabla exista, sea honesta y diga dónde falla.

Al notebook · Parte 2

Casos con imagen, harness y tabla de delta

Escriben ≥5 casos con imagen (uno adversarial) en el formato del eval set. Completan los criterios que faltan en calificar(): campos, válvula, juez VLM o checklist según su patrón. Corren el harness sobre su sistema_multimodal, llenan la tabla de delta con sus números de M3 y exportan el scorecard M4.

CASOS (≥5) → calificar(caso, salida) → harness → tabla M3 vs. M4 → scorecard_m4.csv · S13_Lab_Integracion_Visual.ipynb

Clase grabada: pausen el video y trabajen la parte 2 (~45 min). Vuelvan con la tabla llena para las mini-presentaciones.

Lo que tiene que existir ahora.

- **La tabla de delta, llena** — con sus números de M3 (del scorecard que ya entregaron) y los de M4 de hoy. Si M3 no tenía casos con imagen, su baseline ahí es 0, y se dice.
- **El peor caso con imagen, identificado** — cuál, por qué, y qué harían. Es la fila más valiosa de la tabla.
- **La latencia por caso** — y la respuesta a "¿cuánto cuesta ver?" en segundos.
- **El scorecard exportado** — scorecard_m4.csv en el repo, junto al eval set con imágenes.

Con esto tienen el 80 % de M4. Lo que falta para el domingo: el README de una página y limpiar el repo.

Cuatro minutos por equipo.

No es la demo final.

Cinco cosas, en este orden.

1. **LA PREGUNTA (20 s)**
   La frase de su usuario que necesita ojos. Sin contexto, sin historia.

2. **EL PATRÓN Y EL MODELO (30 s)**
   input / tool / output · qué modelo · dónde se enchufó. Una diapositiva o la celda del notebook.

3. **UN CASO EN VIVO (90 s)**
   Una imagen real del dominio entra, la respuesta sale. Si el Colab no coopera, una captura de la corrida.

4. **LA TABLA DE DELTA (60 s)**
   Sin imagen vs. con imagen vs. latencia. Los números que salieron, no los que querían.

5. **EL FALLO (40 s)**
   El peor caso: qué pasó y qué harían. El equipo que no encontró fallos no buscó.

Cómo se califica.

CRITERIO	PESO	QUÉ SE MIRA
Pipeline multimodal funcionando	3.0	sistema_multimodal responde de punta a punta con UN componente visual real (no un mock), reproducible desde el repo
Justificación del patrón	1.5	La pregunta del usuario, por qué ese patrón y ese modelo, y qué alternativa descartaron. "Ninguno, y por esto" vale si está medido
Eval set con imagen	2.0	≥5 casos con imagen, criterio declarado por caso, al menos uno adversarial, imágenes anonimizadas en el repo
Harness, scorecard y delta	2.5	La misma función de M2 califica los casos nuevos; tabla M3 vs. M4 con latencia; peor caso analizado
Límites y ética	1.0	Etiqueta de "imagen generada" si aplica, válvula para lo que no se ve o no se debe hacer, datos personales fuera del repo

Lo que va más allá de lo visto en clase (ColPali, SAM como tool, un ViT fine-tuneado con LoRA) se valora en la nota, igual que en M1–M3. Entrega tardía: −2 % por día, máximo 7 días.

Cuatro cosas en el repo, en una carpeta m4/.

m4/pipeline
El notebook de hoy terminado o un módulo importable: ver(), sistema_multimodal() y la conexión con su sistema de M3. Debe correr desde cero en Colab.

m4/eval_set
El CSV con los casos con imagen (≥5) y la carpeta de imágenes referenciadas. Sin datos personales: anonimizan o usan imágenes propias.

m4/scorecard_m4.csv + tabla de delta
La salida del harness, caso por caso, y la tabla M3 vs. M4 (con latencia) en el README.

m4/README.md · una página
La pregunta, el patrón, el modelo, dónde se enchufó, la tabla de delta, el peor caso y los límites. Si alguien lee solo esto, entiende M4.

Regla de siempre: la entrega se acumula. M4 es una pieza del proyecto final (S16 · demo day · 5 NOV · 40 %): lo que entreguen el domingo es lo que van a desplegar en S15.

Imágenes de personas, imágenes inventadas.

Las fotos de sus usuarios son datos personales
Caras, documentos, placas, historias clínicas: no van al repo público. Anonimicen, recorten o usen imágenes propias. Y díganle al usuario qué pasa con su foto.

Una imagen generada se marca como generada
Etiqueta visible en la interfaz y en el archivo. En salud, legal y e-commerce, una imagen sintética sin marcar es engaño, no producto.

Ver no es diagnosticar
Un VLM describe; no decide. Salud, vet y agro: la salida es "esto parece", con la válvula, y la decisión es de un humano. Díganlo en el README.

El modelo también tiene sesgos de fábrica
CLIP y SD aprendieron de internet: pieles, acentos, contextos no colombianos. Midan al menos un caso donde eso podría importar en su dominio.

La rúbrica del proyecto final tiene un criterio explícito de limitaciones y ética (5 %). Lo que escriban hoy en el README de M4 es el borrador de eso.

La entrega M4 cierra el domingo.

Con hoy se cierra la Unidad 4: ver (S11), dibujar (S12), integrar y medir (S13). Lo que construyeron en los dos labs es la entrega; lo que falta es el README y limpiar el repo.

Entreguen
m4/ con pipeline, eval set con imágenes, scorecard y README de una página. Domingo 25 OCT, 23:59. Clase grabada: el video de 4 min va en Teams con el mismo plazo.

Recuerden
Las imágenes de personas y documentos no van al repo. La imagen generada se marca. El VLM describe, no diagnostica.

Pregunta para llevarse: de todo lo que ahora hace su sistema, ¿qué es lo primero que un usuario real notaría que tarda demasiado?

La próxima sesión: que quepa y que corra rápido.

El sistema ya hace todo lo que debe. Ahora pesa: un generador, un retriever, un VLM y quizás un modelo de difusión. En S14 empieza el Módulo 5, eficiencia: destilación (un modelo chico que aprende del grande), cuantización (4 bits en vez de 16) y PEFT avanzado. En S15, despliegue y observabilidad. En S16, demo day.

Traigan a S14
La latencia por caso de su scorecard M4 y el nombre del modelo más pesado de su pipeline (parámetros y memoria). Eso es lo qu e vamos a comprimir.

Pregunta para pensar
¿Cuál de sus componentes tardaría demasiado para un usuario real? Esa es la primera candidata a destilar o cuantizar.

S14 es el 22 OCT (M4 sigue abierta hasta el dom 25). S15 el 29 OCT. Demo day S16 el 5 NOV.

Integrar es enchufar y medir.
Lo demás ya lo tenían.

Un componente visual, una función ver(), un enchufe en el sistema de M3, cinco casos con imagen y una tabla que dice si mejoró, si no empeoró y cuánto cuesta. Eso es M4. El domingo se entrega; en S14 empieza la eficiencia.